Wireless
Internet of Things

Project	Indoor Signal Strength Measurements
Name, Matric Number	Muhammad Syahir Irshad Bin Mohd Sharuddin, 6056209
	Muhammad Isma Haikal bin Ismail, 6057561
Date	06.11.2025
Lecturer	Prof. Dr. Jans Werner

Table of Contents

1. Introduction..... 3
2. Hardware Setup 3
3. Measurement Procedure 4
4. Code Overview 4
5. Analysis of Results 6
6. Conclusion 8

1. Introduction

This project investigates indoor WiFi signal strength (RSSI) distribution within the H402 laboratory. An ESP8266 NodeMCU was programmed to perform WiFi scans and extract the RSSI levels of specific access points, including H402 and all eduroam APs (identified individually through their MAC addresses). The measurements were taken at 57 predefined positions following the provided floor plan.

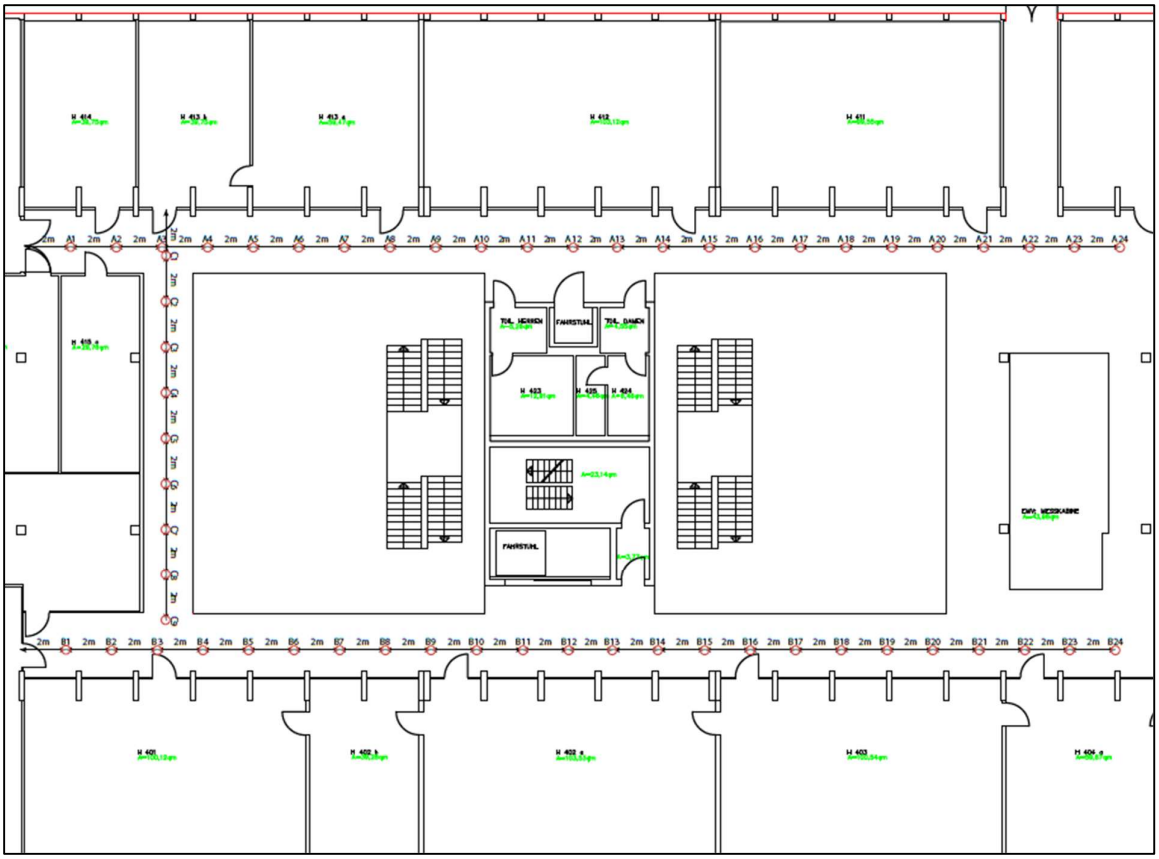
A custom firmware was developed to scan WiFi access points and store the results locally in a CSV file on the ESP8266's LittleFS filesystem. Each measurement was triggered manually via the serial interface using a computer connected to the ESP8266. After completing all measurements, the collected data was exported and analyzed in Excel to determine the path-loss exponent γ.

This report describes the hardware setup, software implementation, measurement process, and final RSSI-based analysis.

2. Hardware Setup

The ESP8266 NodeMCU served as the measurement device due to its built-in WiFi module and ease of programming with the Arduino framework. The ESP8266 was powered directly via USB from the laptop during all measurements.

Measurements were collected inside the university according to the grid provided:



3. Measurement Procedure

At each measurement point:

1. Move the laptop + ESP8266 to the location.
2. Place ESP8266 at the correct spot (A1, A2, …, C9).
3. Type *n* into the serial monitor.
4. Wait for scan to finish.
5. Move to the next marked point.
6. 57 measurements were recorded this way.
7. Type *d* into the serial monitor to print out the data into a table form.
8. The output between *BEGIN CSV* and *END CSV* was copied into a data.csv file.
9. The file was opened in Excel and integrated into the provided Excel reference sheet.

4. Code Overview

At first, we need to initialise and declare the needed variable.

```
#include <Arduino.h>
#include <ESP8266WiFi.h>
#include <FS.h>
#include <LittleFS.h>

// ==================== CONFIG ====================

// Flash button on ESP8266 (GPIO0 / D3), uses internal pull-up
// This is the built-in button on the NodeMCU board
const int BUTTON_PIN = D3;  // GPIO0 - flash button
const int LED_PIN = D4;   // Built-in LED (GPIO2)

// File on LittleFS
const char* DATA_FILE = "/rssi_data.csv";

// Target SSIDs
const char* SSID_H402   = "H402";
const char* SSID_EDUROAM = "eduroam";   // lowercase as you requested
```

An array of characters are declared indicating various points on the 4th floor in which the measurements of the RSSI are taken of the insisted Wi-Fi networks.

```
// Measurement points: A1..A24, B1..B24, C1..C9
const char* measurementPoints[] = {
  // A row
  "A1","A2","A3","A4","A5","A6","A7","A8","A9","A10","A11","A12",
  "A13","A14","A15","A16","A17","A18","A19","A20","A21","A22","A23","A24",
  // B row
  "B1","B2","B3","B4","B5","B6","B7","B8","B9","B10","B11","B12",
  "B13","B14","B15","B16","B17","B18","B19","B20","B21","B22","B23","B24",
  // C row
  "C1","C2","C3","C4","C5","C6","C7","C8","C9"
};
```

A function to record the data on each point is provided below. In this function you can see that typing “n” on the serial monitor will begin the measurements.

Adding to the function is the focus of the Wi-Fi searching system to only measure and store the signal strengths for the networks with SSID’s “H402” and “eduroam”:

```
// Do one measurement at the current point
void measureAtPoint(uint8_t index) {
  if (index >= NUM_POINTS) {
    Serial.println("All measurement points already processed.");
    return;
  }

  const char* label = measurementPoints[index];

  Serial.println();
  Serial.println("========================================");
  Serial.printf("Measuring at point %s (%u/%u)\n",
                label, index + 1, NUM_POINTS);
  Serial.println("Scanning WiFi...");

  // active scan, include hidden networks
  int n = WiFi.scanNetworks(/*async=*/false, /*show_hidden=*/true);
  Serial.printf("Scan done. %d networks found.\n", n);

  if (n <= 0) {
    Serial.println("No networks found.");
    return;
  }

  for (int i = 0; i < n; i++) {
    String ssid  = WiFi.SSID(i);
    int32_t rssi = WiFi.RSSI(i);
    uint8_t* bssid = WiFi.BSSID(i);  // 6 bytes

    // Only store H402 and eduroam
    if (ssid == SSID_H402 || ssid == SSID_EDUROAM) {
      appendMeasurementRow(label, ssid.c_str(), bssid, rssi);
    }
  }
```

Another few functions to print out the stored data into the indicated table format by typing “d”. Also, the function to clear out the store data by typing “c” when needed.

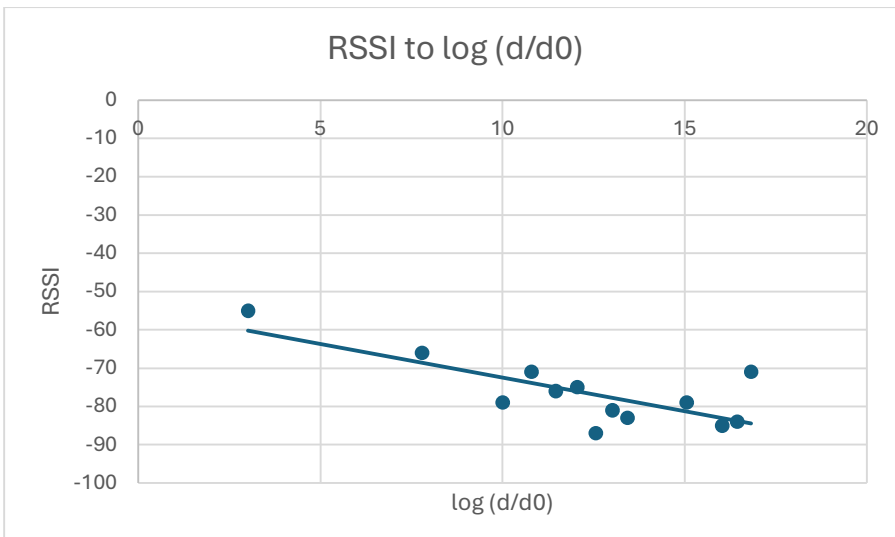
```
// Dump full CSV to serial
void dumpCsv() {
  if (!LittleFS.exists(DATA_FILE)) {
    Serial.println("No CSV file yet.");
    return;
  }

  File f = LittleFS.open(DATA_FILE, "r");
  if (!f) {
    Serial.println("ERROR: Cannot open CSV for read.");
    return;
  }

  Serial.println();
  Serial.println("----- BEGIN CSV -----");
  while (f.available()) {
    Serial.write(f.read());
  }
  Serial.println("----- END CSV -----");
  f.close();
}

// Clear CSV file
void clearCsv() {
  if (LittleFS.exists(DATA_FILE)) {
    LittleFS.remove(DATA_FILE);
    Serial.println("CSV file deleted.");
  } else {
    Serial.println("No CSV file to delete.");
  }
}
```

5. Analysis of Results

The CSV was imported into the reference Excel template. Then the values of RSSI, appointed distance and logarithmic distance are extracted and calculated from the data specifically of ‘H402’ at each available point as can be seen on the Excel.

Some points had no H402 reading, because the device sometimes did not detect the H402 AP at that position. These data are extracted to create a graphical presentation as can be seen below (here only A1-A24), so that the calculation of γ can be done.

Point	Distance	RSSI	log(d/d0)
A1	2	-55	3.0103
A3	6	-66	7.781513
A5	10	-79	10
A6	12	-71	10.79181
A7	14	-76	11.46128
A8	16	-75	12.0412
A9	18	-87	12.55273
A10	20	-81	13.0103
A11	22	-83	13.42423
A16	32	-79	15.0515
A20	40	-85	16.0206
A22	44	-84	16.43453
A24	48	-71	16.81241



Added to the graph is a linear regression of the plotted points that helps for the calculation of γ.

From the linear regression between RSSI and $\log_{10}(d)$, the slope of the fitted line is approximately **–17.83**, which corresponds to a path-loss exponent:

$$\gamma = -\frac{slope}{10} \approx 1.8$$

This value can now be compared with the theoretical free-space model where the exponent is:

$$\gamma_{free\ space} = 2$$

A γ value slightly below 2 means the signal in the H402 environment decays slower than it would in open free space. This is typical for indoor corridors, where walls, floors, and ceilings cause waveguide effects that channel the radio waves and reduce attenuation. As a result:

- reflections help the signal travel farther,
- the path-loss curve becomes less steep,
- and the received power remains higher than expected for the same distance in free space.

6. Conclusion

A functioning RSSI data acquisition system was successfully developed using the ESP8266. The system performed WiFi scans on command and stored measurement results in a structured CSV format suitable for analysis. Manual triggering via serial input allowed controlled data collection for all 57 measurement points.

The results imported into Excel enabled calculation of the path-loss exponent γ, providing insight into radio propagation characteristics within the H402 facility. The project demonstrates the effectiveness of the ESP8266 as a compact, low-cost measurement tool for indoor wireless analysis.